System Software
Lab Assignment - 1

Name : Naishadh Rana

Enrollment Number : U23CS014

---

1. Write a program to convert Regular Expression to DFA.

```
#include <stdio.h>
#include <string.h>

#define MAX 128

typedef struct {
   int from;
   char sym;
   int to;
} Transition;

Transition T[MAX];
int tcount = 0;
int final_state = 0;

void add_tr(int from, char sym, int to) {
   T[tcount].from = from;
   T[tcount].sym = sym;
   T[tcount].to = to;
   tcount++;
}

void build_dfa(const char *re) {
```

```
    int cur = 0;
    tcount = 0;
    int n = (int)strlen(re);

    for (int i = 0; i < n; ++i) {
        char ch = re[i];
        char nxt = (i + 1 < n) ? re[i + 1] : '\0';

        if (ch == '*' || ch == '+' || ch == '|') continue;

        if (nxt == '*') {
            add_tr(cur, ch, cur);
            i++;
        } else if (nxt == '+') {
            add_tr(cur, ch, cur + 1);
            add_tr(cur + 1, ch, cur + 1);
            cur += 1;
            i++;
        } else {
            add_tr(cur, ch, cur + 1);
            cur += 1;
        }
    }
    final_state = cur;
}

int validate(const char *s) {
    int state = 0;
    for (int i = 0; s[i]; ++i) {
        char ch = s[i];
        int moved = 0;
        for (int j = 0; j < tcount; ++j) {
            if (T[j].from == state && T[j].sym == ch) {
                state = T[j].to;
                moved = 1;
                break;
            }
        }
        if (!moved) return 0;
    }
    return state == final_state;
}
```

```c
void print_dfa(void) {
   printf("\nDFA\n");
   printf("%-8s %-8s %-8s\n", "State", "Input", "Next");
   printf("------------------------\n");
   for (int i = 0; i < tcount; ++i) {
       printf("q%-7d %-8c q%-7d\n", T[i].from, T[i].sym, T[i].to);
   }
   printf("------------------------\n");
   printf("Start: q0\nFinal: q%d\n", final_state);
}

int main() {
   char re[64], s[64], c;
   printf("Enter regex: ");
   scanf("%63s", re);

   build_dfa(re);
   print_dfa();

   do {
       printf("\nTest a string? (y/n): ");
       scanf(" %c", &c);
       if (c == 'y' || c == 'Y') {
           printf("String: ");
           scanf("%63s", s);
           if (validate(s)) printf("ACCEPTED\n");
           else printf("REJECTED\n");
       }
   } while (c == 'y' || c == 'Y');
   return 0;
}
```

Output:

```
Enter regex: a+b*

DFA
State    Input    Next
------------------------
q0       a        q1
q1       a        q1
q1       b        q1
------------------------
Start: q0
Final: q1

Test a string? (y/n): y
String: a
ACCEPTED

Test a string? (y/n): y
String: ab
ACCEPTED

Test a string? (y/n): y
String: b
REJECTED

Test a string? (y/n): y
String: abbbbba
ACCEPTED

Test a string? (y/n): n
```

2. Write a program to detect tokens in c program.

I will be considering keywords, identifiers, string literals, operators, numbers and special characters (like bracket open close, semi colon) as tokens

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char str[]) {
   char keywords[12][10] = {"int", "float", "if", "else", "while",
                            "for", "return", "void", "char", "main",
                            "printf", "scanf"};
   for(int i = 0; i < 12; i++) {
       if(strcmp(keywords[i], str) == 0)
           return 1;
   }
   return 0;
}

int main() {
   char input[100], token[20];
   int i = 0, j = 0;

   printf("Enter a C statement: ");
   fgets(input, 100, stdin);
   printf("\nTokens identified:\n");
   printf("-------------------\n");

   while(input[i] != '\0') {
       j = 0;
       if(input[i] == '"') {
           i++; // used to only consider the string without the quotes
           while(input[i] != '\0' && input[i] != '"') {
               token[j++] = input[i++];
           }
           token[j] = '\0';
           if(input[i] == '"') i++; // skip closing quote
           printf("String Literal: %s\n", token);
       }
```

```
        else if(isalpha(input[i]) || input[i] == '_') {
            while(isalnum(input[i])) {
                token[j++] = input[i++];
            }
            token[j] = '\0';
            if(isKeyword(token))
                printf("Keyword: %s\n", token);
            else
                printf("Identifier: %s\n", token);
        }
        else if(isdigit(input[i])) {
            while(isdigit(input[i])) {
                token[j++] = input[i++];
            }
            token[j] = '\0';
            printf("Number: %s\n", token);
        }
        else if((input[i] == '>' && input[i+1] == '=') ||
                (input[i] == '<' && input[i+1] == '=') ||
                (input[i] == '=' && input[i+1] == '=') ||
                (input[i] == '!' && input[i+1] == '=')) {
            printf("Operator: %c%c\n", input[i], input[i+1]);
            i += 2;
        }
        else if(input[i] == '+' || input[i] == '-' || input[i] == '*' ||
                input[i] == '/' || input[i] == '=' || input[i] == '<' ||
                input[i] == '>') {
            printf("Operator: %c\n", input[i]);
            i++;
        }
        else if(input[i] == ';' || input[i] == ',' || input[i] == '(' ||
                input[i] == ')' || input[i] == '{' || input[i] == '}') {
            printf("Special Symbol: %c\n", input[i]);
            i++;
        }
        else {
            i++;
        }
    }
    return 0;
}
```

Outputs:

```
Enter a C statement: int sum = 1+123

Tokens identified:
-------------------
Keyword: int
Identifier: sum
Operator: =
Number: 1
Operator: +
Number: 123
```

```
Enter a C statement: if (num > 50) {printf("Start the show");}

Tokens identified:
-------------------
Keyword: if
Special Symbol: (
Identifier: num
Operator: >
Number: 50
Special Symbol: )
Special Symbol: {
Keyword: printf
Special Symbol: (
String Literal: Start the show
Special Symbol: )
Special Symbol: ;
Special Symbol: }
```

3. Write a C program to recognize strings under:

- ‘b+a*’
- ‘a*b+a*’

```c
#include <stdio.h>
#include <string.h>

int recognizePattern1(char str[]) {
   int i = 0, bCount = 0;

   while(str[i] == 'b') {
       bCount++;
       i++;
   }

   if(bCount == 0)
       return 0;

   while(str[i] == 'a')
       i++;

   return (str[i] == '\0');
}

int recognizePattern2(char str[]) {
   int i = 0, bCount = 0;

   while(str[i] == 'a')
       i++;

   while(str[i] == 'b') {
       bCount++;
       i++;
   }

   if(bCount == 0)
       return 0;

   while(str[i] == 'a')
```

```
        i++;

    return (str[i] == '\0');
}

int main() {
    char str[100];

    printf("Enter a string: ");
    scanf("%s", str);

    printf("\nPattern Recognition Results:\n");
    printf("----------------------------\n");

    if(recognizePattern1(str))
        printf("Pattern 'b+a*': ACCEPTED\n");
    else
        printf("Pattern 'b+a*': REJECTED\n");

    if(recognizePattern2(str))
        printf("Pattern 'a*b+a*': ACCEPTED\n");
    else
        printf("Pattern 'a*b+a*': REJECTED\n");

    return 0;
}
```

Output:

```
Enter a string: bba

Pattern Recognition Results:
----------------------------
Pattern 'b+a*': ACCEPTED
Pattern 'a*b+a*': ACCEPTED
```

```
Enter a string: aaa

Pattern Recognition Results:
----------------------------
Pattern 'b+a*': REJECTED
Pattern 'a*b+a*': REJECTED
```

```
Enter a string: abba

Pattern Recognition Results:
----------------------------
Pattern 'b+a*': REJECTED
Pattern 'a*b+a*': ACCEPTED
```